基于通信信号的运动检测

汇报人 周晓俊 何衍宁 张安泰 夏天

2024/12/25

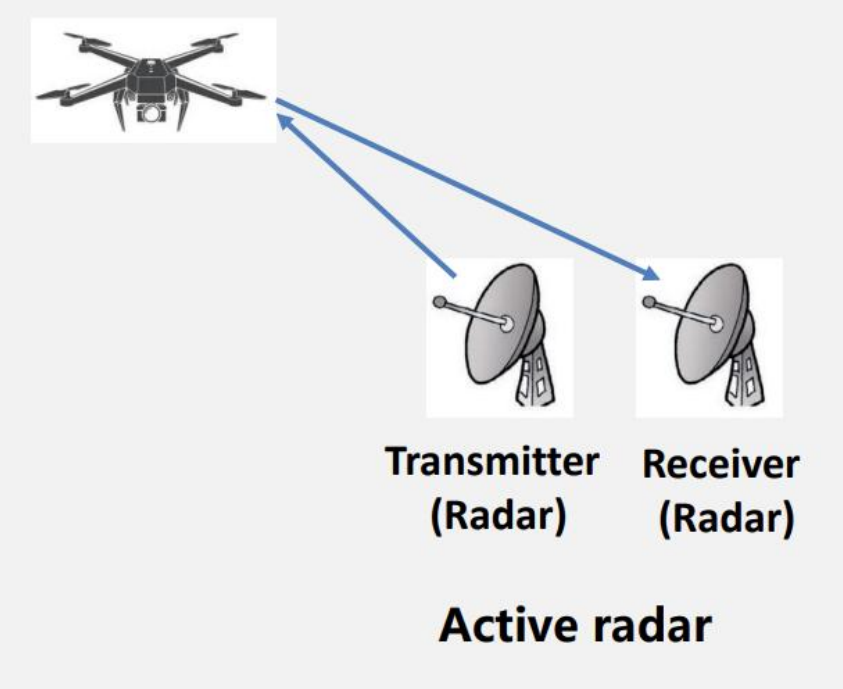
目录

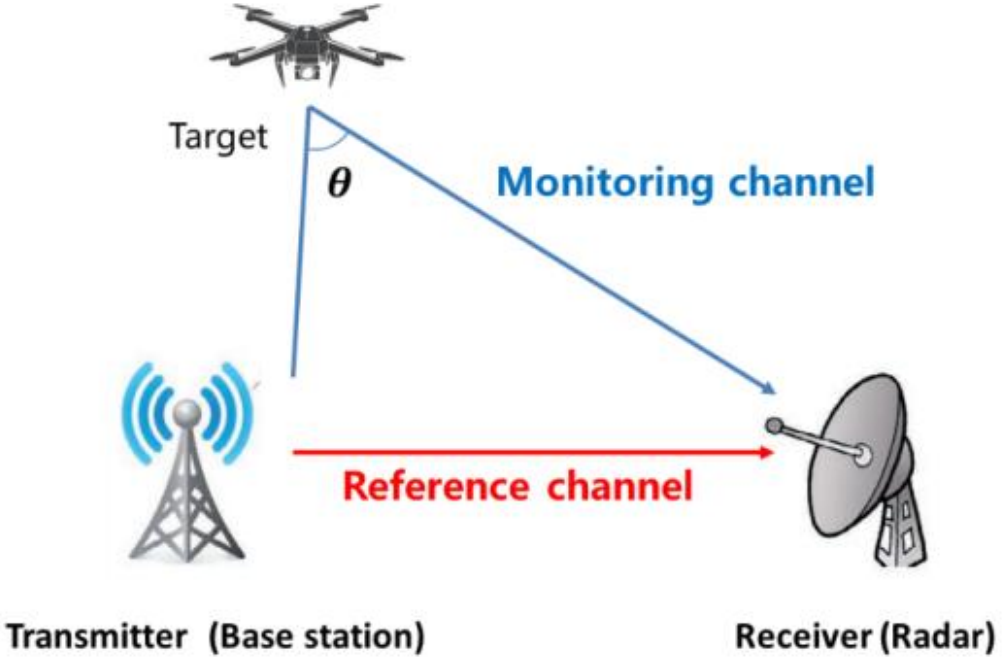
01 项目背景

02 关键概念

03 实验结果


01
项目背景





无源雷达: 与主动雷达不同，其不依赖于自身的信号发射器，而是利用外部信号源（如通信基站、电视广播发射塔、卫星等）作为信号发射源。在这种系统中，基站或其他外部发射机提供参考信号，而雷达设备则充当接收机，接收来自不同目标物体的散射信号。由于目标物体的反射表面会改变信号的传播路径，这些反射信号会与参考信号之间存在一定的时间差和频率差。

数学模型

$$y_{ref}(t) = \alpha x(t - \tau_r),$$

$$y_{sur}(t) = \beta x(t - \tau_s) e^{j2\pi f t}$$

设发射信号为$x(t)$,
参考信道传输的参考信号为$y_{ref}(t)$,

监视信道传输的监视信号为$y_{sur}(t)$,

α和τ_r分别表示参考信道的衰减系数和时延;

β和τ_s分别表示监视信道的衰减系数和时延。

被动雷达接收来自基站发射的参考信号，与此同时，接收机也会接收到从目标折射回来的信号。通过计算从两个通道采样的信号之间的时间差τ和频率差f_{d}，可以估计目标的距离和多普勒频移。

目标距离 $= \tau \times \mathrm{c}$

目标速度 $= \dfrac{\lambda \times f_{\mathrm{d}}}{2cos(\theta/2)}, \theta = 90$

其中c表示电磁波传播速度，本实验中$\mathrm{c} = 3 \times 10^8 m/s$, λ表示载波波长，本例中，载波频率为 2123MHz。时间差τ等于采样偏移量除以采样率，本例中，参考信号和监视信号的采样率均为 25MS/s。

多普勒效应

多普勒频移是由信号源与观测器（或接收器）之间的相对运动引起的一 种现象。当信号源和接收器相对运动时，接收到的信号频率会发生变化。这 种频率的变化是由于波源发出的波的传播与接收机的运动状态之间相互作用 所造成的。具体来说，当信号源和接收器之间存在相对运动时，接收器感知 到的信号频率会因为运动而发生偏移。当信号源和接收器靠近时，波的波长 会变短，从而使接收器接收到的频率增加，这种现象被称为“蓝移”或正频移。 当信号源和接收器远离时，波的波长会变长，接收到的频率减少，这种现象 被称为“红移”或负频移。波源振动后会发出一个波，而接收机由于相对运动而接收到不同的频率。

$$f' = \left(\frac{v \pm v_0}{v \mp v_s}\right) f$$

三．项目目标

利用被动雷达对目标监测得到的二十组数据，分析并得到被测目标的距离与速度。利用被动雷达对目标进行监测时，通常通过接收外部信号源（如基站、 广播塔或卫星等）发射的电磁波信号，并分析目标反射回来的信号来获取目标的动态信息。

X

Part 02

关键概念

数字下变频

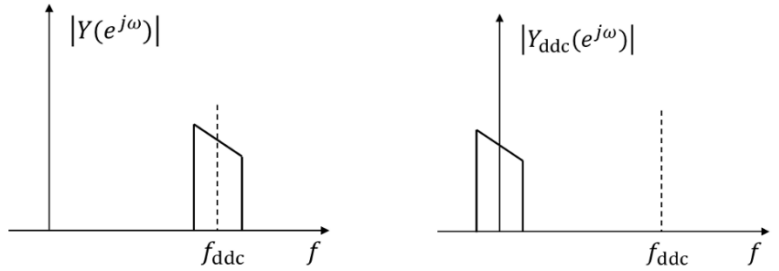
数字下变频指的是将高频信号转换为基带信号的过程。
本项目中，对采集的信号需要进行f_o=-3MHz的数字下变频处理。

设原始信号$y(n)$的频谱为$Y(e^{j\omega})$，经过数字下变频后，得到的信号$y_{\mathrm{ddc}}(n)$为，

$$y_{\mathrm{ddc}}(n) = y(n)e^{-j2\pi f_o n}$$

根据离散时间傅里叶变换的频移特性，数字下变频处理之后的频谱$Y_{\mathrm{ddc}}(e^{j\omega})$为，

$$Y_{\mathrm{ddc}}(e^{j\omega}) = Y(e^{j2\pi(f+f_o)n})$$



（a）数字下变频之前　　（b）数字下变频之后

图 3：数字下变频

经过数字下变频处理，其信号频谱的实际变化为**向右平移**3MHz。

变频后可以将信号所需要部分移动到低频区域，便于后续处理。

具体代码如下：

```
% DDC

f_ddc = -3*1e6;

seq_sur_ddc_t = seq_sur .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);

seq_sur_ddc_f = 20*log10(abs(fftshift(fft(seq_sur_ddc_t))));
```

低通滤波器

低通滤波器对信号中低于某一频率f_{cut}（低通滤波器的截止频率）的成分均能以常值增益通过，去掉高频部分。

本项目中，我们采用butter函数编写LPF，并通过filter函数对信号进行滤波处理。

具体代码如下：

```
% LPF

[b, a] = butter(20, bandwidth/(0.5*f_s));

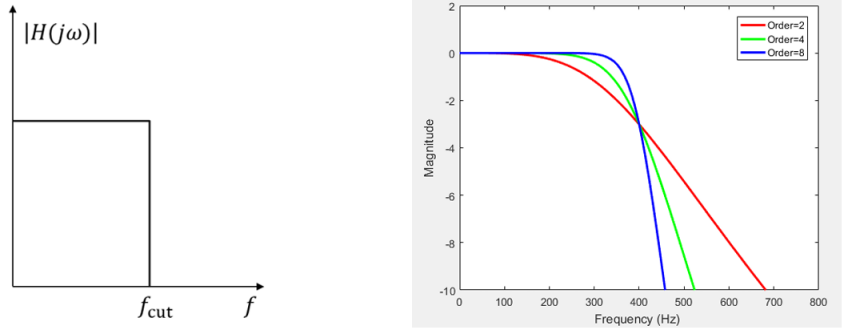
seq_sur_lpf_t = filter(b, a, seq_sur_ddc_t);

seq_sur_lpf_f = 20*log10(abs(fftshift(fft(seq_sur_lpf_t))));
```

$$|H(\omega)| = \frac{1}{\sqrt{1+\left(\frac{\omega}{\omega_c}\right)^{2n}}}$$

式中，ω_c为通带截止频率，n为滤波器的阶数。巴特沃斯滤波器有以下幅频特性：

- 通带内具有最大平坦的幅频特性。
- 随频率的增大，平滑单调下降。
- 在通带中，是理想的单位响应，在阻带中响应为零。
- 在截止频率处即当$\omega = \omega_c$时，$|H(\omega_c)| = 1/\sqrt{2}$，有半功率频率，即有3dB衰减。



（a）理想低通滤波器　　（b）巴特沃斯低通滤波器

图 4：低通滤波器

巴特沃斯低通滤波器阶数n越高，滤波器过渡带越陡峭，期中过渡带的陡峭成都正比于滤波器的阶数。
随着阶数n越高，响应越接近矩形，过渡带越窄，所以高阶巴特沃斯低通滤波器的幅频响应特性就越近于理想低通滤波器

X

Part 03

实验内容

数字下变频分析

01 数字下变频输出波形的时域特性，包括幅度和相位信息。

02 数字下变频输出频谱的分析，展示信号的频率分布。

03 实验数据与理论预测的对比，验证数字下变频的准确性。

（4）数字下变频

软件无线电系统采集的基站信号的中心频率是 2123MHz，带宽是 25MHz，包含 2130-2135MHz(5M 带宽)的基站信号和 2110-2130MHz(20M 带宽)的信号。本案例选取 20MHz 带宽信号作为分析对象，需要对采集的信号做$f_o = -3\text{MHz}$的数字下变频处理。数字下变频，这里指的是从频带信号中提取复基带信号的过程。

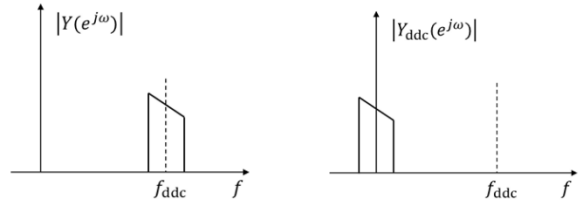
设原始信号$y(n)$的频谱为$Y(e^{j\omega})$，经过数字下变频后，得到的信号$y_{\text{ddc}}(n)$为，

$$y_{\text{ddc}}(n) = y(n)e^{-j2\pi f_o n}$$

根据离散时间傅里叶变换的频移特性，数字下变频处理之后的频谱$Y_{\text{ddc}}(e^{j\omega})$为，

$$Y_{\text{ddc}}(e^{j\omega}) = Y(e^{j2\pi(f+f_o)n})$$

频移后的结果如图 3 所示。需要注意的是，本案例中$f_o = -3\text{MHz}$，频谱实际向右移动3MHz。



（a）数字下变频之前　　（b）数字下变频之后

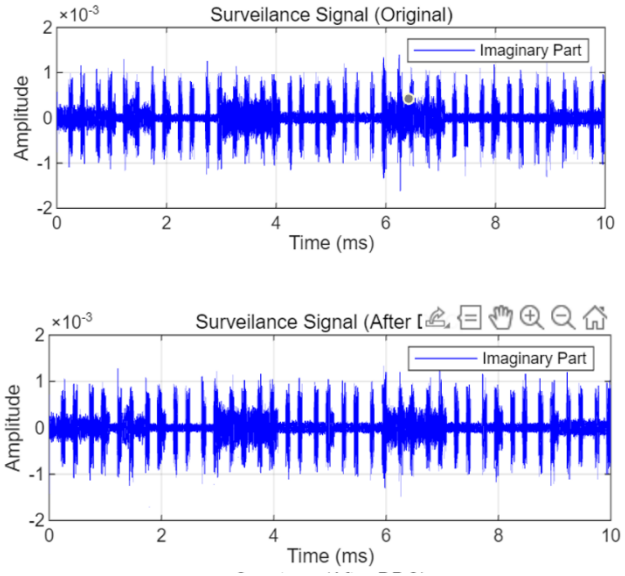
图 3：数字下变频

数字下变频分析

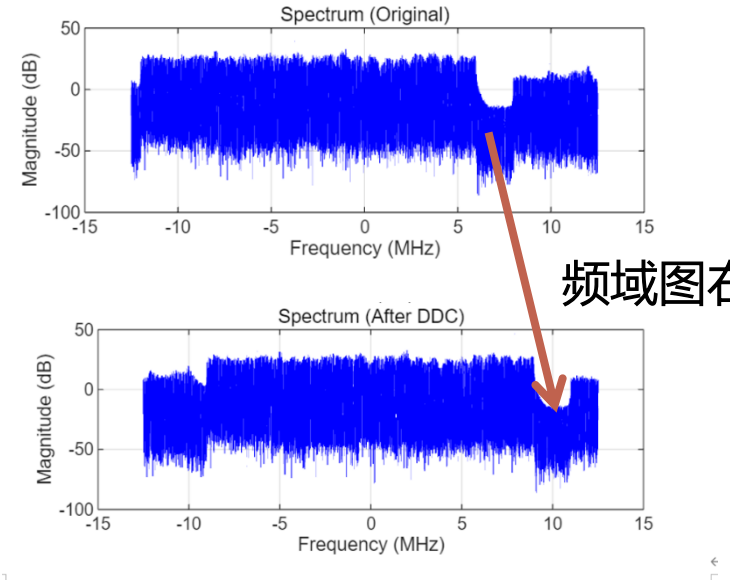
本次实验中软件无线电系统采集的基站信号的中心频率时2123MHz，带宽25MHz，
信号包含2130-2135MHz（5M带宽）、2110-21230MHz（20M带宽）。

①接收2110-2130MHz和2130-2135MHz两个频段的基带信号。
②在信号处理前，需要滤除2130-2135MHz(5M带宽)的基站信号，保留2110-2130MHz(20M带宽)的信号。
③将2110-2130MHz(20M带宽)信号**变频到零中频**，再通过低通滤波器滤掉高频信号。

数字下变频输出的时域图与频域图：（以data_1为例，f_o=-3MHz）



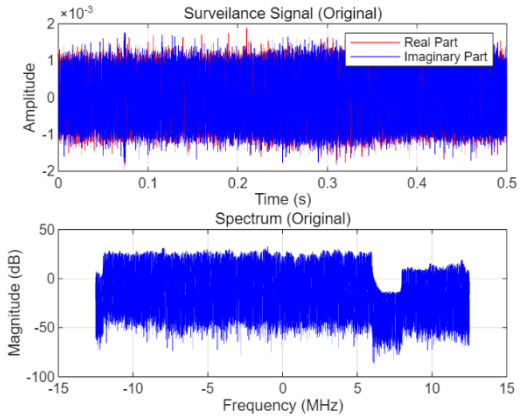
监测信号时域数字下变频输出图



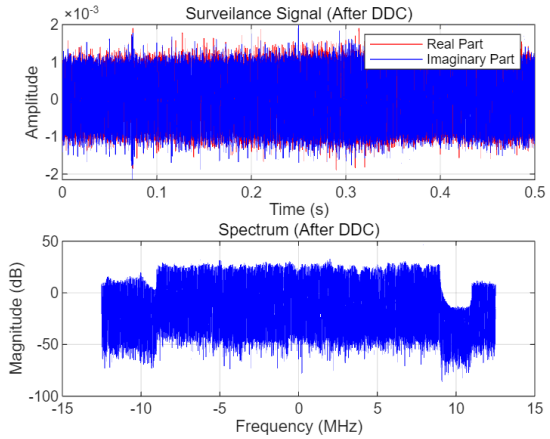
监测信号频域数字下变频输出图

时域图+频谱图

监测信号

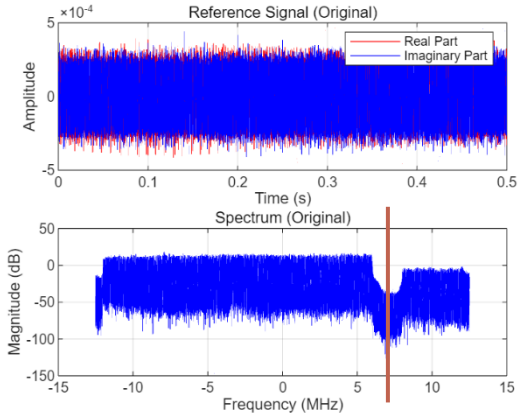


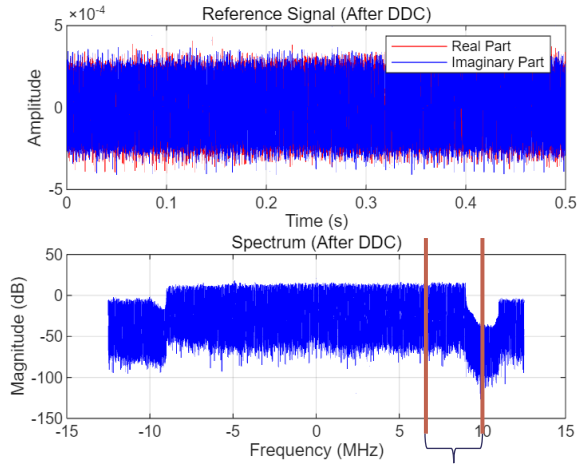
原始



数字下变频

参考信号





3MHz

注释：

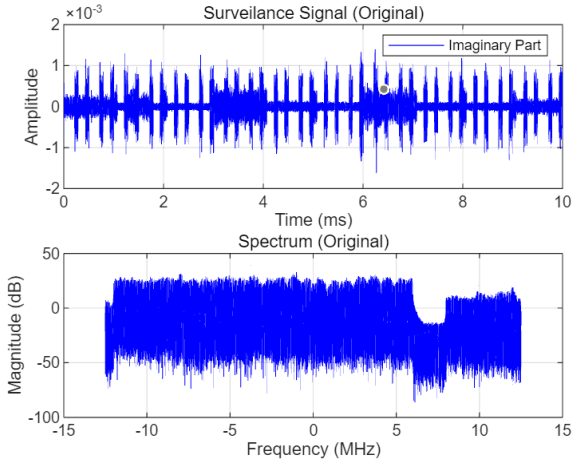
1. **一到二行**分别为监测信号、参考信号的原始输出与数字下变频输出的时域图以及频谱图

问题：

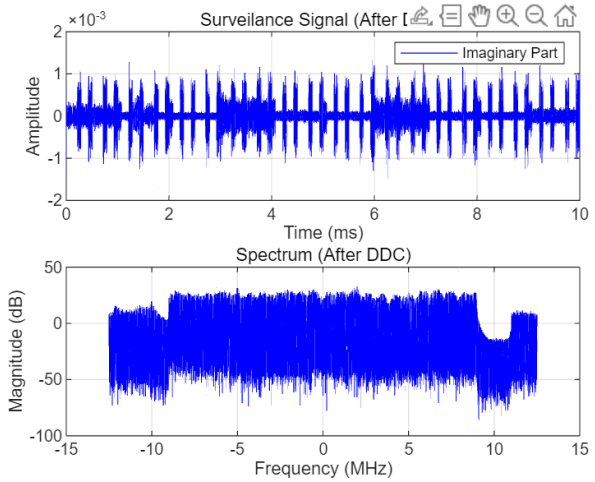
1. 监测信号与参考信号（无论是原始输出或是数字下变频输出），其时域图极其**复杂**，难以进行进一步分析。

时域图+频谱图

监测信号

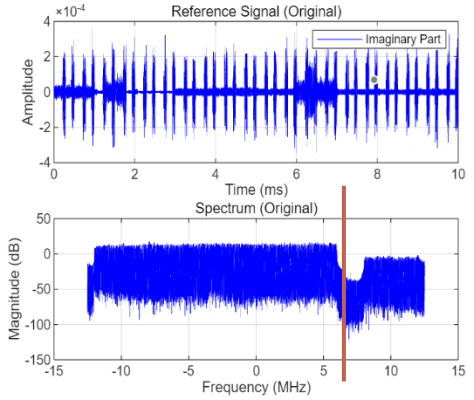


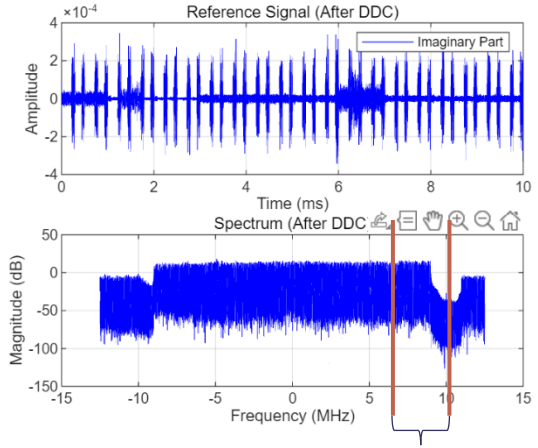
原始



数字下变频

参考信号





3MHz

注释：

1. 频谱图与示例频谱图较为**接近**
2. 前文问题通过限制坐标轴取值，以及仅对实部/虚部进行绘图，得出相对好**分析**的图像。
3. 幅度单位已全部转化为**分贝**

低通滤波分析

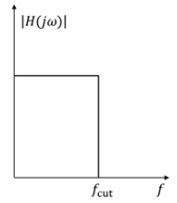
01 数字滤波器的验证，验证低通滤波是否符合要求。

02 低通滤波输出波形的时域特性，包括幅度和相位信息。

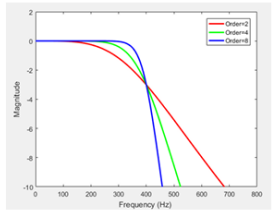
03 低通滤波输出频谱的分析，展示信号的频率分布。

（5）低通滤波器

低通滤波器对信号中低于某一频率f_{cut}的成分均能以常值增益通过，f_{cut}称为低通滤波器的截止频率。理想低通滤波器的频率相应如图 4(a)所示。



（a）理想低通滤波器



（b）巴特沃斯低通滤波器

图 4：低通滤波器

巴特沃斯(Butterworth)滤波器是一种典型的电子滤波器，在通信、测量等领域里已有广泛应用。巴特沃斯低通滤波器幅频响应表达式为，

$$|H(\omega)| = \frac{1}{\sqrt{1+\left(\frac{\omega}{\omega_c}\right)^{2n}}}$$

式中，ω_c为通带截止频率，n为滤波器的阶数。巴特沃斯滤波器有以下幅频特性：

- 通带内具有最大平坦的幅频特性。
- 随频率的增大，平滑单调下降。
- 在通带中，是理想的单位响应，在阻带中响应为零。
- 在截止频率处即当$\omega = \omega_c$时，$|H(\omega_c)| = 1/\sqrt{2}$，有半功率频率，即有 3dB 衰减。

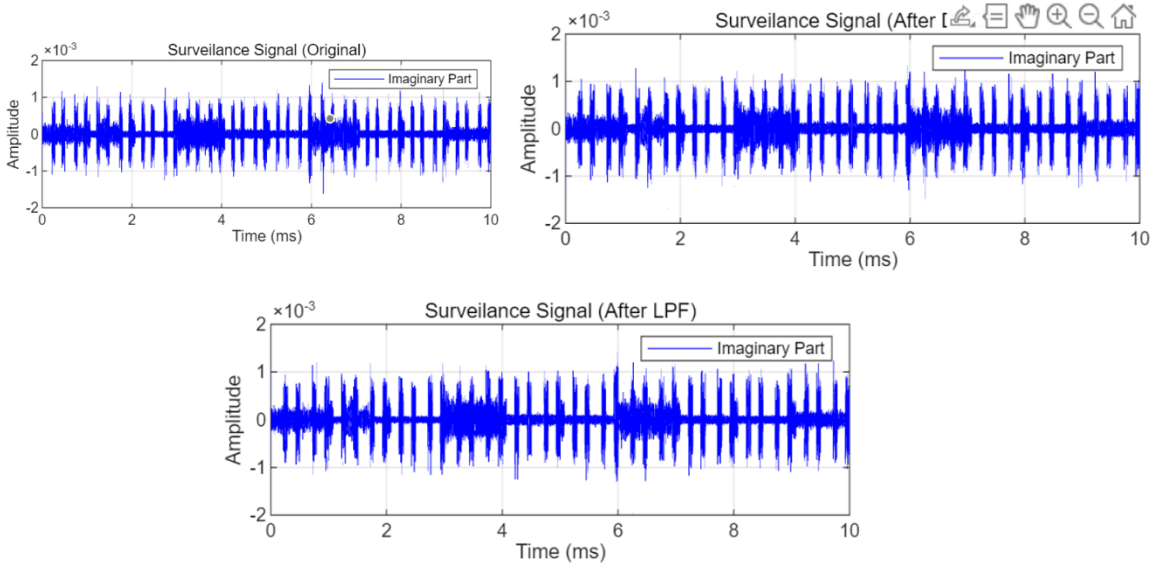
巴特沃斯低通滤波器的幅频特性曲线如图 4（b）所示。由图中可以看出，巴特沃斯低通滤波器阶数n越高，滤波器过渡带越陡峭，也即过渡带的陡峭程度正比于滤波器的阶数。随着阶数n越高，响应越接近矩形，过渡带越窄，所以高阶巴特沃斯低通滤波器的幅频响应特性就越近于理想低通滤波器。

低通滤波分析

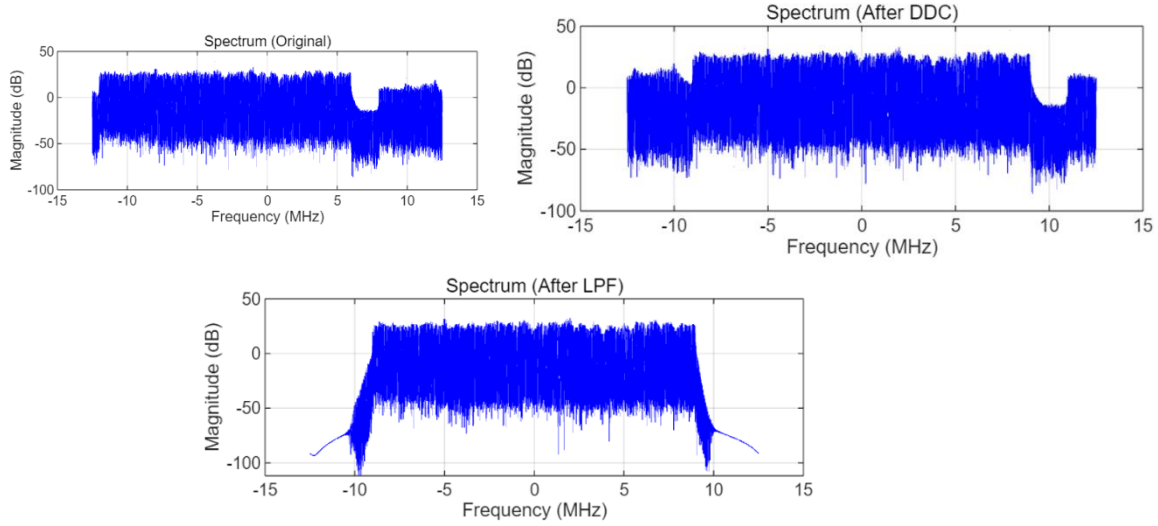
本次实验中软件无线电系统采集的基站信号的中心频率时2123MHz，带宽25MHz，
信号包含2130-2135MHz（5M带宽）、2110-21230MHz（20M带宽）。

①接收2110-2130MHz和2130-2135MHz两个频段的基带信号。
②在信号处理前，需要滤除2130-2135MHz(5M带宽)的基站信号，保留2110-2130MHz(20M带宽)的信号。
③将2110-2130MHz(20M带宽)信号变频到零中频，再通过**低通滤波器滤掉高频信号**。

数字下变频输出的时域图与频域图：（以data_1为例）



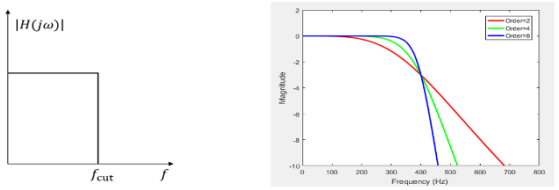
监测信号时域低通滤波输出图



监测信号频域低通滤波输出图

低通滤波分析

巴特沃斯（Butterworth）滤波器是一种典型的电子滤波器。
我们采用butter函数设计低通滤波器。



（a）理想低通滤波器　　（b）巴特沃斯低通滤波器

图 4：低通滤波器

具体代码如下：

```
addpath('data');

load(sprintf('data\\data_%d.mat', 1));

bandwidth = 9e6;

[b, a] = butter(20, bandwidth/(0.5*f_s));

[H,f_H] = freqz(b,a,4096,f_s);

figure;

plot(f_H*1e-6,20*log10(abs(H)));

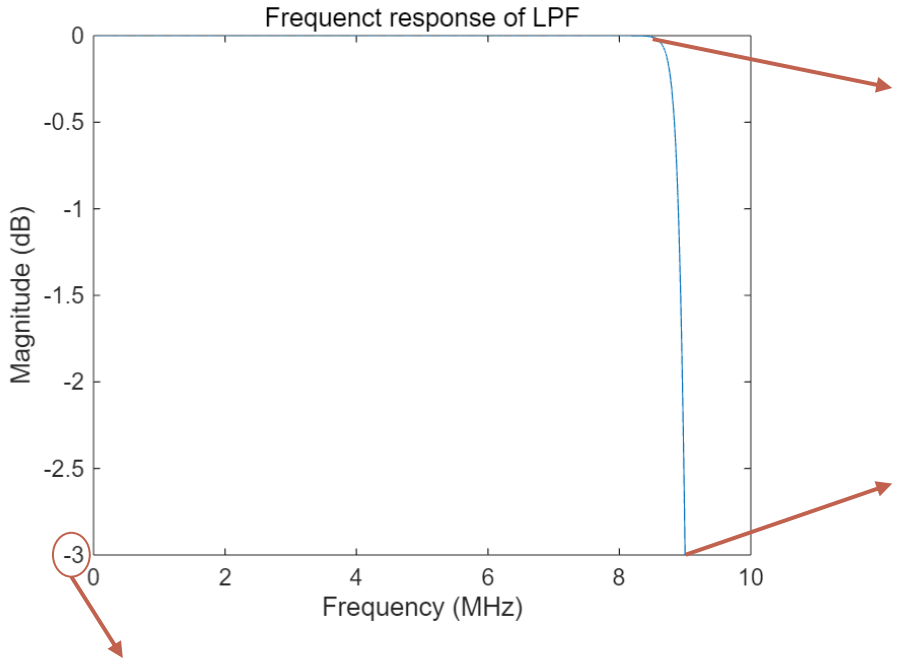
xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

ylim([-3,0]);

title('Frequenct response of LPF');
```

采用freqz函数画出该低通滤波器的频率响应，
以进行验证该滤波器为一个合适的LPF。



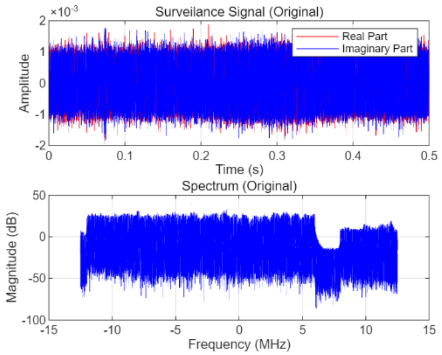
频率范围为0~8MHz内，信号的强度基本保持不变。

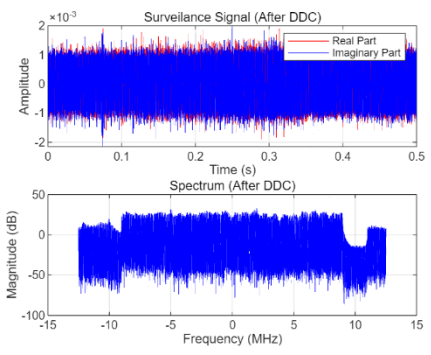
频率高于9MHz的区域为阻带，信号被显著削弱。

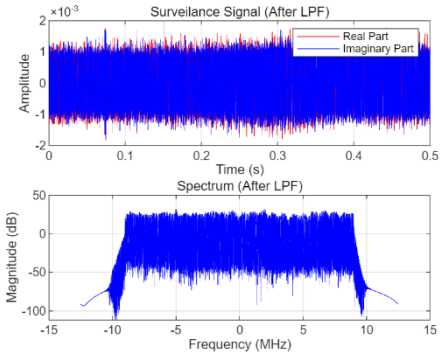
信号强度降低

时域图+频谱图

监测信号

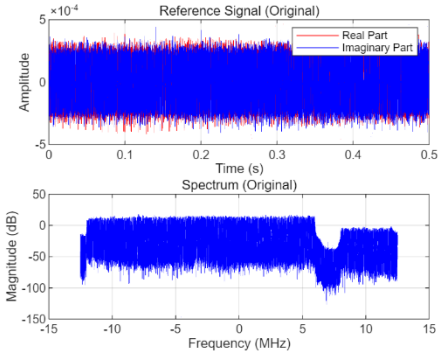


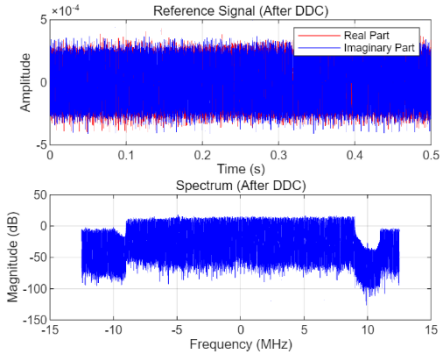


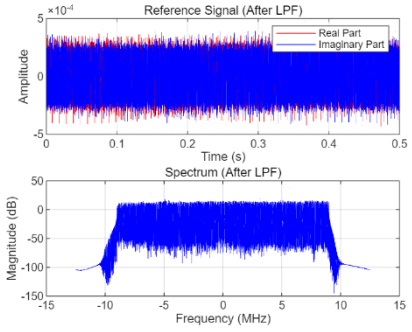


原始　　数字下变频　　低通滤波

参考信号







注释：

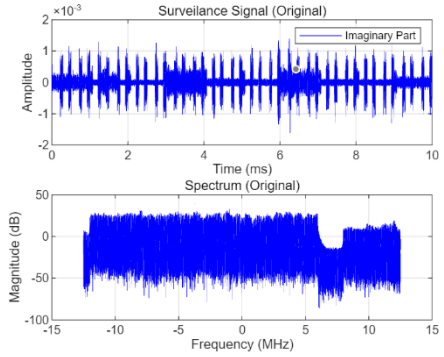
1. **一到三行**分别为监测信号、参考信号的原始输出、数字下变频、低通滤波输出的时域图以及频谱图

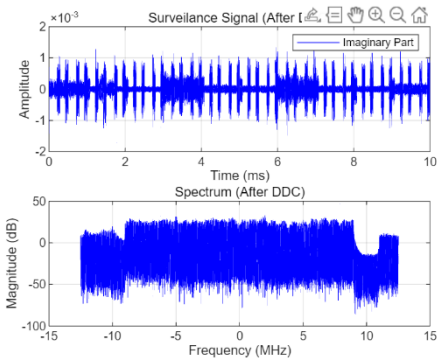
问题：

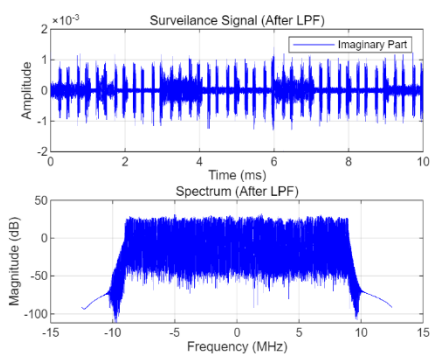
1. 监测信号与参考信号，其时域图极其**复杂**，难以进行进一步分析。

时域图+频谱图

监测信号

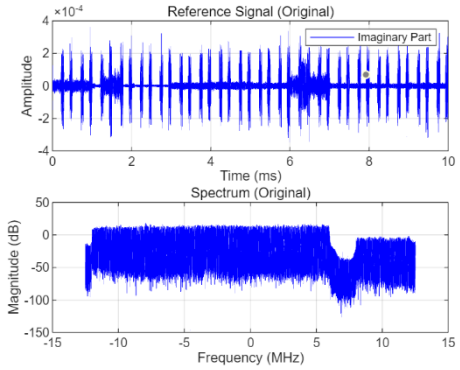


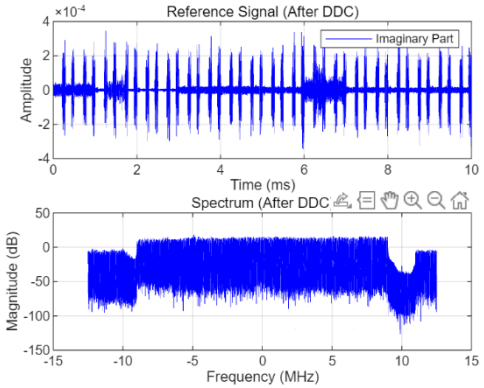


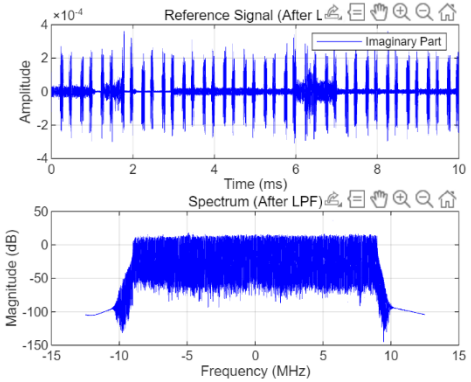


原始　　数字下变频　　低通滤波

参考信号







注释：

1. 频谱图与示例频谱图较为**接近**
2. 前文问题通过限制坐标轴取值，以及仅对实部/虚部进行绘图，得出相对好**分析**的图像。
3. 幅度单位已全部转化为**分贝**

总结：

1. 从第二列可以看出，信号片段发生了平移，对应**数字下变频**处理。
2. 从第三列可以看出，信号片段保留了靠近中心的部分，对应**低通滤波**处理。

定义模糊函数

$$Cor(\tau) = \int_{t}^{t+T} y_{surv}(t) y_{ref}^{*}(t-\tau) e^{-j2\pi f_D t} dt \Longrightarrow Cor(\tau) = \sum_{n=0}^{N-1} y_{surv}[nT_s]\, y_{ref}^{*}[nT_s - \tau]\, e^{-j2\pi f_D n T_s}$$

y_{surv}是监视信号，y_{surv}是参考信号

τ是时延，最小值为$1/f_s$，光速为c，距离最小分度值为c/f_s，为12m，运动物体距离在100m以内，因此τ取1~6，

f_D是多普勒频移，物体运动平均速度为4m/s左右，因此f_D取$-40\sim40$Hz

由于采集的信号是离散信号，计算机也只能处理离散信号，因此首先要将模糊函数公式进行离散化，如上方公式所示。

第一次尝试

$$Cor(\tau) = \int_{t}^{t+T} y_{surv}(t) y_{ref}^{*}(t-\tau) e^{-j2\pi f_D t} dt \Longrightarrow Cor(\tau) = \sum_{n=0}^{N-1} y_{surv}[nT_s]\, y_{ref}^{*}[nT_s - \tau]\, e^{-j2\pi f_D n T_s}$$

尝试使用三层for循环分别遍历时延τ，多普勒频移f_D，信号长度n次求和

运行速度非常慢，每一个信号文件要运行5-6分钟，完整跑一次程序接近三小时！

分析：频繁的进出for循环会消耗很多时间

```
y_sur = seq_sur;
y_ref_conjugate = conj(seq_ref);
cors = zeros(6,41);
tau = 0 : 1 : 6;
fd = -40 : 2: 40;
for i = 1 : 1 : 6
    for j = 1 : 1 : 41
        for k = 1 : 1 : length(seq_ref)
            cors(i,j)=cors(i,j) + y_sur(k) * y_ref_conjugate
                    * exp(-2i * pi * fd(j) * (k-1) * (1 / f_s));
        end
    end
end
```

第二次改进

参考老师代码：
使用一次取模和一次取整的数学运算代替掉两次for循环

运行速度有很大的提升，每一个0.5s的文件进行一次模糊函数遍历时间缩短为一分钟左右，跑一次完整程序大概需要25分钟

```
num_loop = length(array_sample_shift) * length(array_Doppler_frequency);
t_axis = 0:1/f_s:(duration*f_s-1)/f_s;
temp = zeros(1,num_loop);

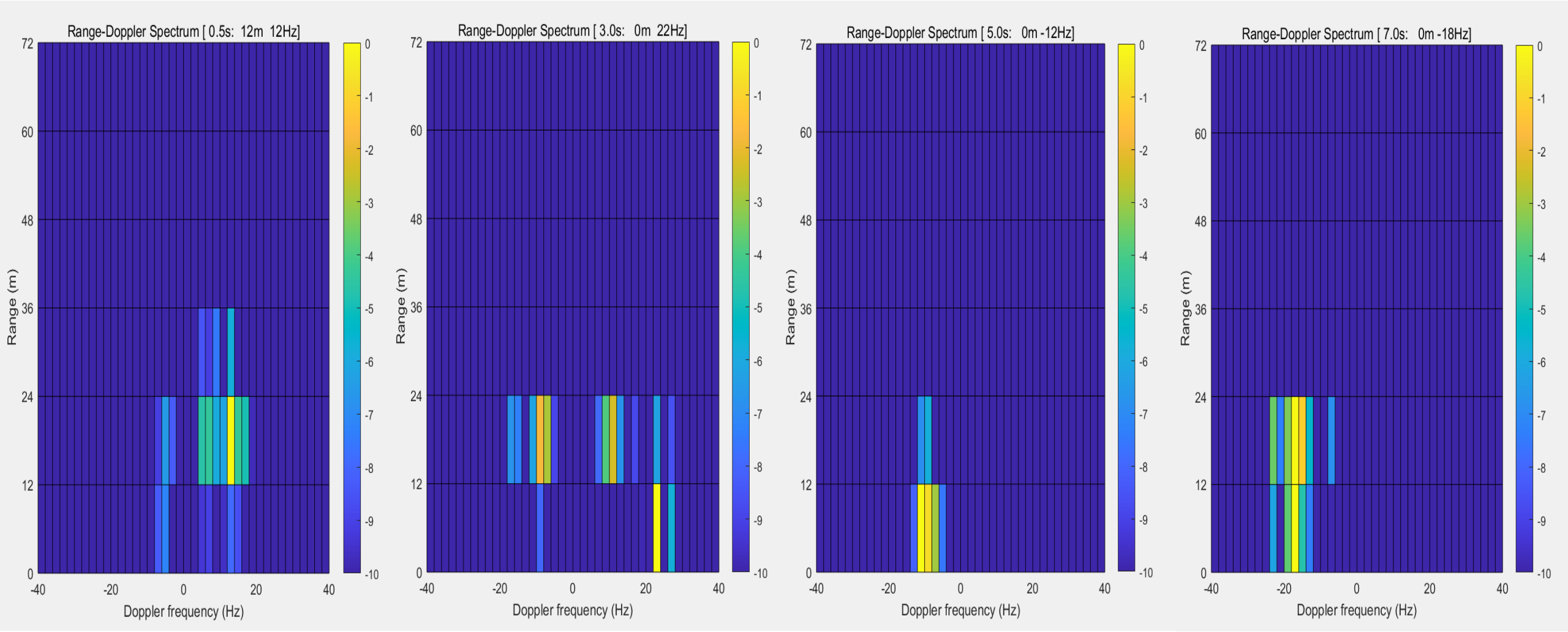
for idx_RD = 1:num_loop
    idx_sample_shift = ceil(idx_RD/length(array_Doppler_frequency));
    idx_f_d = mod(idx_RD-1,length(array_Doppler_frequency))+1;
    sample_shift = array_sample_shift(idx_sample_shift);
    f_d = array_Doppler_frequency(idx_f_d);

    temp(1,idx_RD) = sum(seq_sur_lpf(1,1+sample_shift:end)
                         .*conj(seq_ref_lpf(1,1:end-sample_shift))
                         .*exp(-1i*2*pi*f_d*t_axis(1+sample_shift:end)));

    A_TRD_dwt(idx_start_time, idx_sample_shift, idx_f_d) = abs(temp(1,
idx_RD));
end
```

作图

四个时间段的RD图示例



第三次改进

注意到，离散化后的模糊函数公式与DTFS公式相似,于是令

$$x(n) = y_{surv} .* y_{ref}^{*}$$

$$\frac{k}{N} = f_D$$

使用一个for循环遍历时延 τ ，对$x(n) = y_{surv} .* y_{ref}^{*}$使用fft计算傅里叶变换并使用fftshift将零频移至中心，结果长度为$f_s/2$，取零频正负41个信号整体带入A_TRD中。

$$Cor(\tau) = \sum_{n=0}^{N-1} y_{surv}[nT_s]\, y_{ref}^{*}[nT_s - \tau]\, e^{-j2\pi f_D n T_s}$$

$$X(k) = \sum_{n=0}^{N-1} x(n) \cdot e^{-j\frac{2\pi}{N}kn}$$

```
for idx_sample_shift = 1:length(array_sample_shift)
    sample_shift = array_sample_shift(idx_sample_shift);

    signal_part = seq_sur_lpf(1, 1+sample_shift:end)
                  .* conj(seq_ref_lpf(1, 1:end-sample_shift));

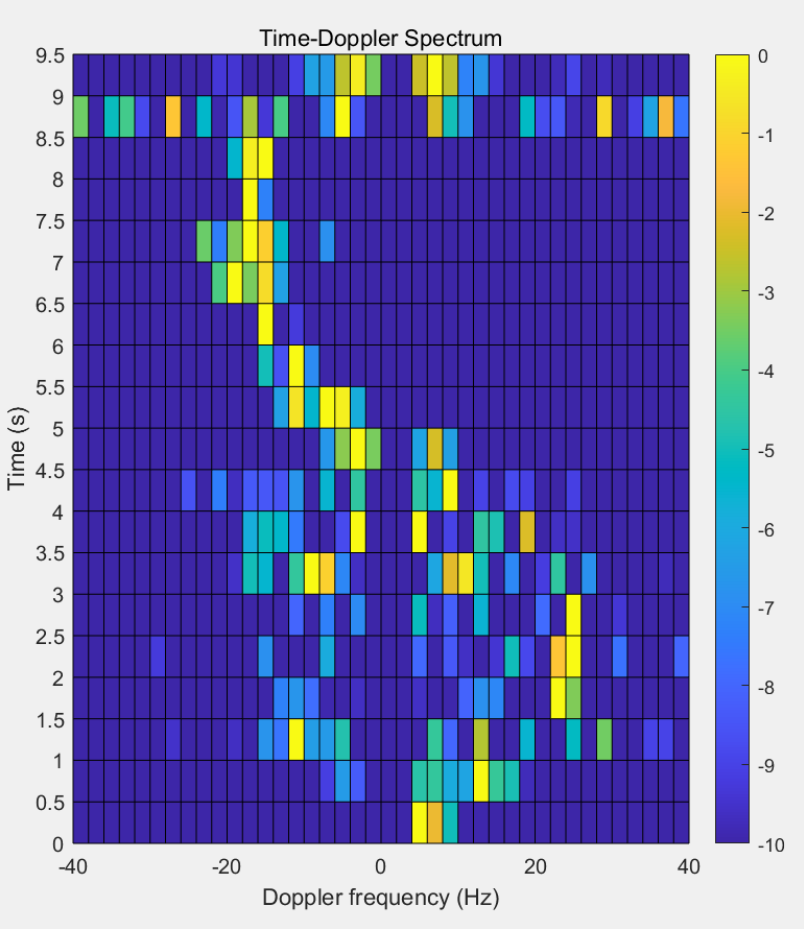
    fft_result = fftshift(fft(signal_part));

    A_TRD(idx_start_time, idx_sample_shift,:) =
    fft_result(((length(fft_result)+1)/2-20):1:((length(fft_result)+1)/2+20));
end
```

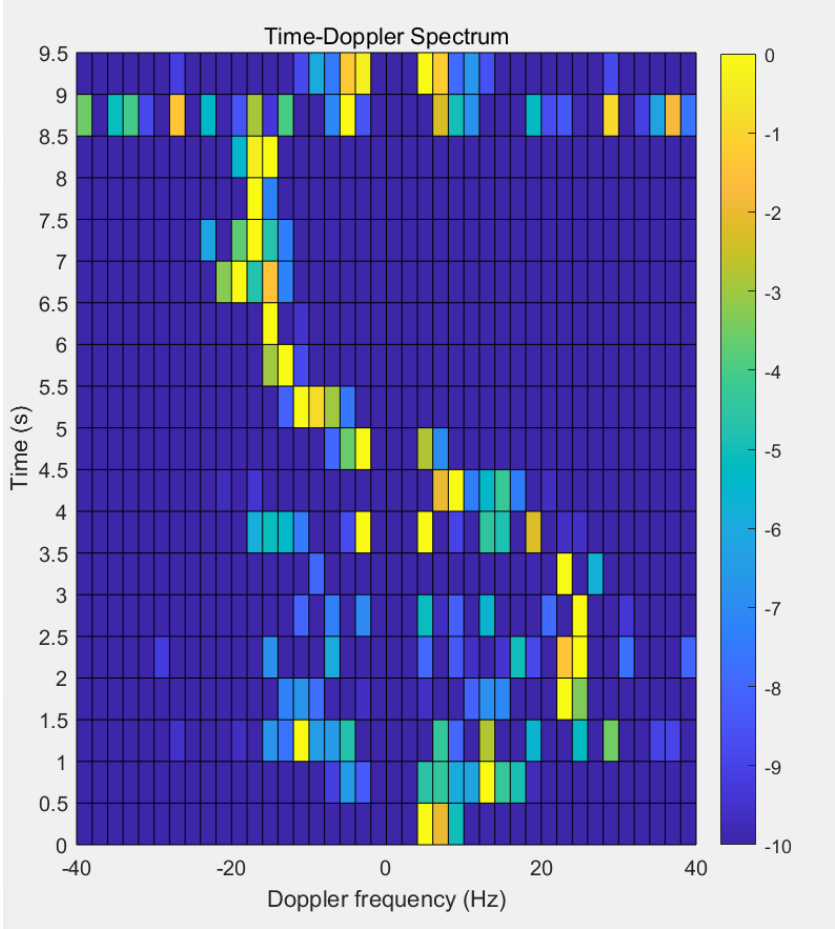
使用fft优化后的模糊函数计算每一个0.5s文件只需要6秒左右，运行一次完整程序只需要4分钟不到，运行速度提升了接近一个数量级。

验证

作出使用fft算法的结果图，发现与遍历算法的结果图有差异



FFT

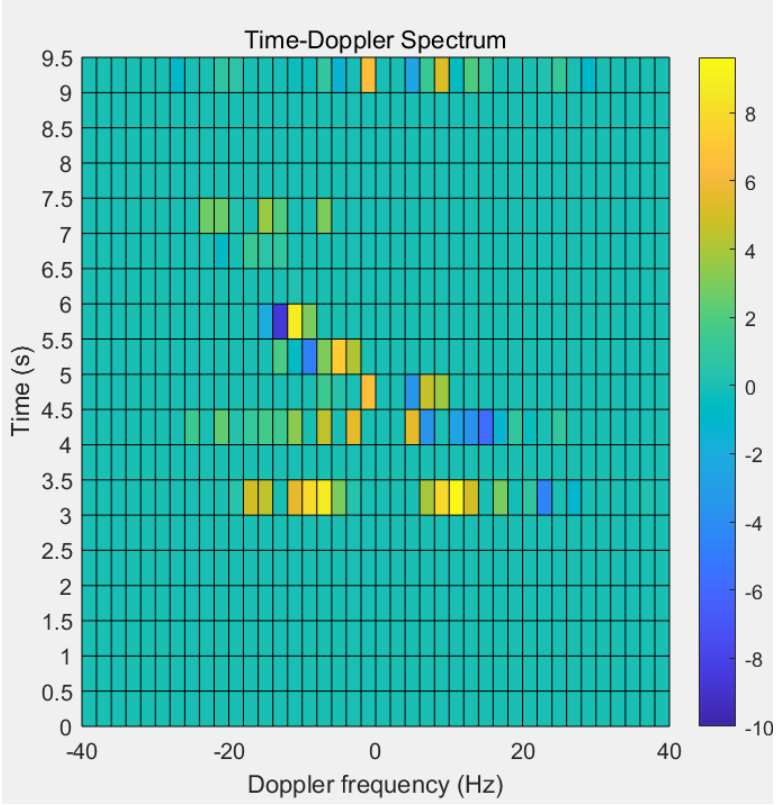


DWT

验证

作出fft算法和原遍历算法下结果的差值图

```
[meshgrid_Doppler, meshgrid_start_time] =
meshgrid(array_Doppler_frequency, [array_start_time,
array_start_time(end)]);
plot_A_TD = zeros(length(array_start_time),
length(array_Doppler_frequency));
plot_A_TD = plot_A_TD_fft - plot_A_TD_dwt;
surf(meshgrid_Doppler, meshgrid_start_time, plot_A_TD);
view(0, 90); colorbar; xlim([array_Doppler_frequency(1),
array_Doppler_frequency(end)]);
ylim([array_start_time(1), array_start_time(end)]);
xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)]);
yticks([array_start_time(1):0.5:array_start_time(end)]);
xlabel('Doppler frequency (Hz)'); ylabel('Time (s)');
title('Time-Doppler Spectrum');
```



从图中可以看出有些时间段存在误差，有些时间段却没有误差，误差的在-10~10之间

分析与修正

对误差产生的原因猜测：
1、模糊函数原公式与FFT公式不完全匹配，需要额外乘上补偿信号
2、频率轴在定义的过程中发生了偏移或拉伸

经过长时间的尝试与检查，发现A_TRD的值在时延为偶数时两种算法完全相同，为奇数时却有较大误差，后续作图函数只取了每个时间段中模糊函数值最大的时延列，因此就会出现取到奇数列有误差，偶数列无误差的情况。
而时延奇偶会影响模糊函数中相乘信号的长度，进而影响fft结果的长度的奇偶，影响其计算结果，只要在奇数时在信号末尾补一个零就可以消除较大的误差！

```
for idx_sample_shift = 1:length(array_sample_shift)
    sample_shift = array_sample_shift(idx_sample_shift);

    signal_part = seq_sur_lpf(1, 1+sample_shift:end)
                    .* conj(seq_ref_lpf(1, 1:end-sample_shift));

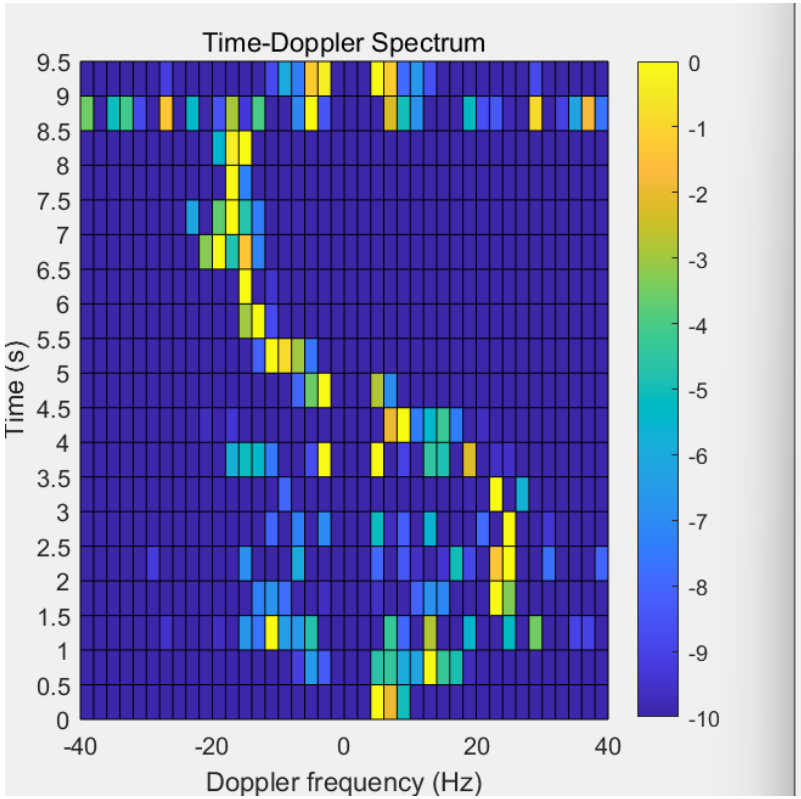
    if mod(length(signal_part), 2) == 0
        signal_part = [signal_part, 0];
    end

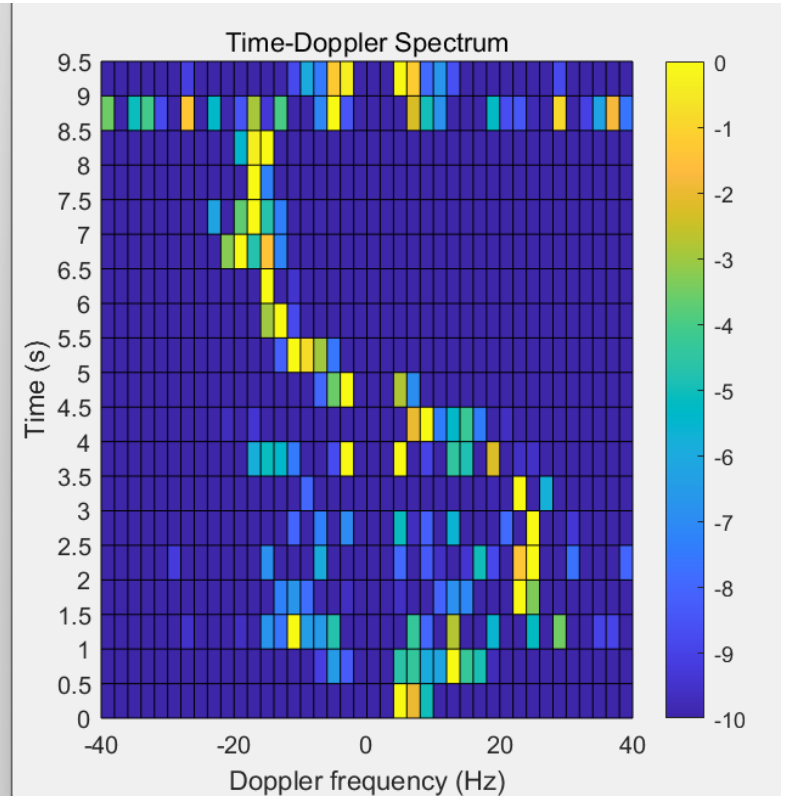
    fft_result = fftshift(fft(signal_part));

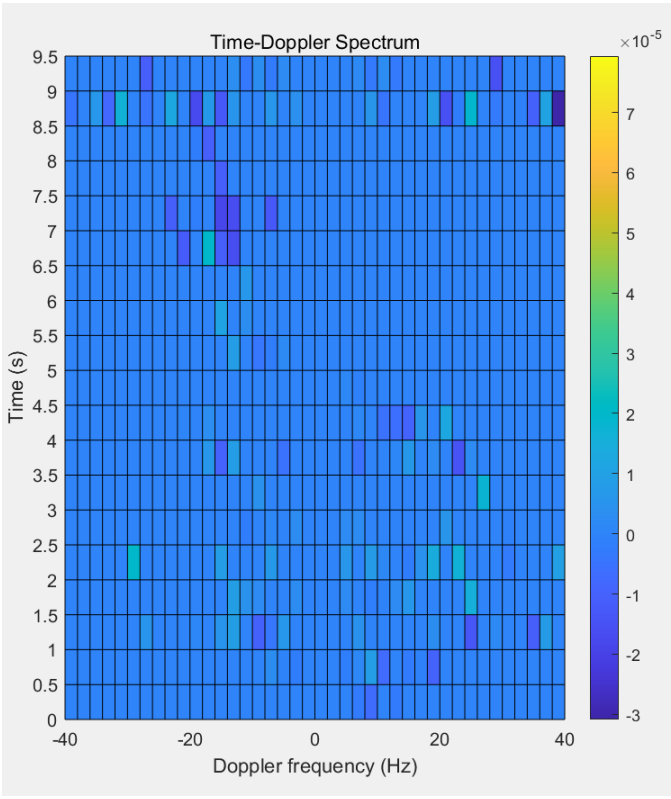
    A_TRD(idx_start_time, idx_sample_shift,:) =
    fft_result(((length(fft_result)+1)/2-20):1:((length(fft_result)+1)/2+20));
end
```

验证

再次作出结果图和误差图







可以看到在修正fft奇偶后两种算法的误差已经降到10e-5数量级，可以忽略不计

时间对比

```
[stat] Index of start time: 1 / 20.
[stat]   Read data file.
[stat]   Downconvert.
[stat]   LPF.
[stat]   Ambiguity processing.
历时 6.202627 秒。
[stat]   Ambiguity processing.
历时 56.115044 秒。
[stat] Index of start time: 2 / 20.
[stat]   Read data file.
[stat]   Downconvert.
[stat]   LPF.
[stat]   Ambiguity processing.
历时 5.365857 秒。
[stat]   Ambiguity processing.
历时 61.212005 秒。
```

单个时间段模糊函数遍历与fft算法时间对比

```
[stat] Index of start time: 20 / 20.
[stat]   Read data file.
[stat]   Downconvert.
[stat]   LPF.
[stat]   Ambiguity processing.
历时 231.233677 秒。
```

fft算法一轮完整程序耗时（包括DDC、LPF、模糊函数计算、作图）

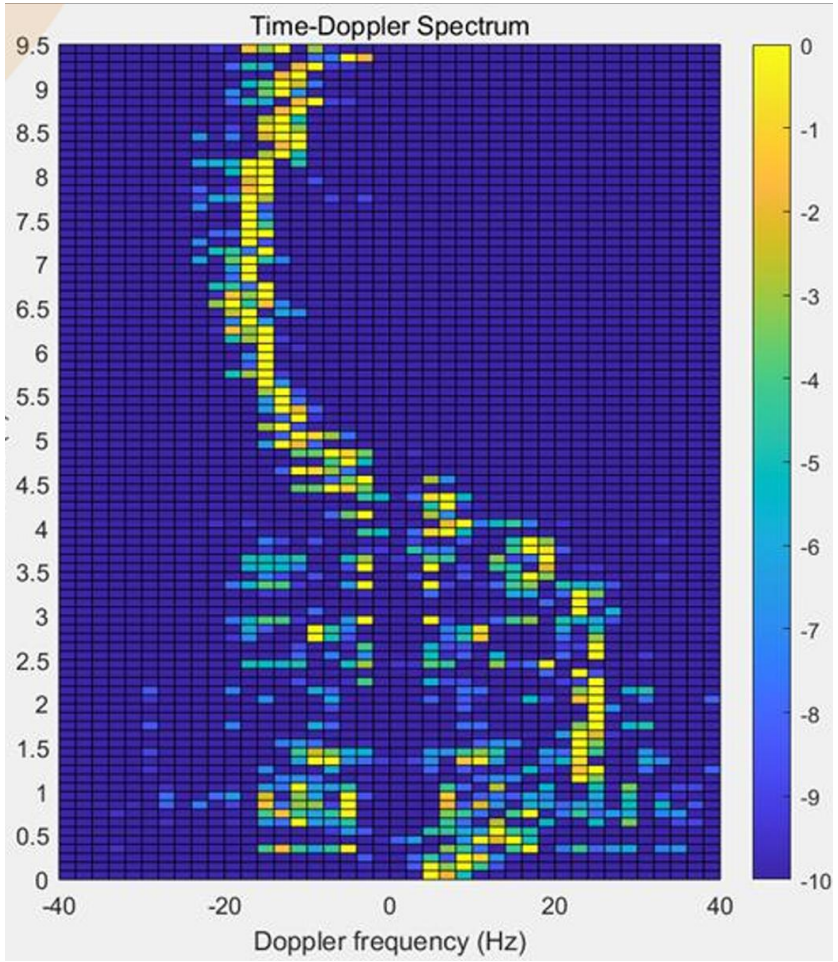
0.1s分割尝试

原分析以0.5s为一个时间段，尝试将0.5s细分为0.1s再做分析
首先将array_start_time = [0:0.5:9.5]; 改为array_start_time = [0:0.1:9.9];
然后将文件读取部分代码改为

```
for idx_start_time = 1:length(array_start_time)
start_time = array_start_time(idx_start_time);
file_idx = floor(start_time / 0.5) + 1; % 确定文件编号
offset = mod(start_time, 0.5) * f_s; % 计算在文件内的偏移量
% 加载当前及相邻文件数据
fprintf('[stat] Index of start time: %d / %d.\n', idx_start_time, length(array_start_time))
fprintf('[stat] Read data files.\n')
load(sprintf('data/data_%d.mat', file_idx));
% 提取所需的时间段数据
seq_ref_window = seq_ref(1, 1 + offset:offset + duration / N * f_s);
seq_sur_window = seq_sur(1, 1 + offset:offset + duration / N * f_s);
%% Digital downconvert %数字下变频
fprintf('[stat] Downconvert.\n')
seq_ref_ddc = seq_ref_window.*exp(-1i*2*pi*f_ddc*[0:duration / N *f_s-1]/f_s);
seq_sur_ddc = seq_sur_window.*exp(-1i*2*pi*f_ddc*[0:duration / N *f_s-1]/f_s);
```

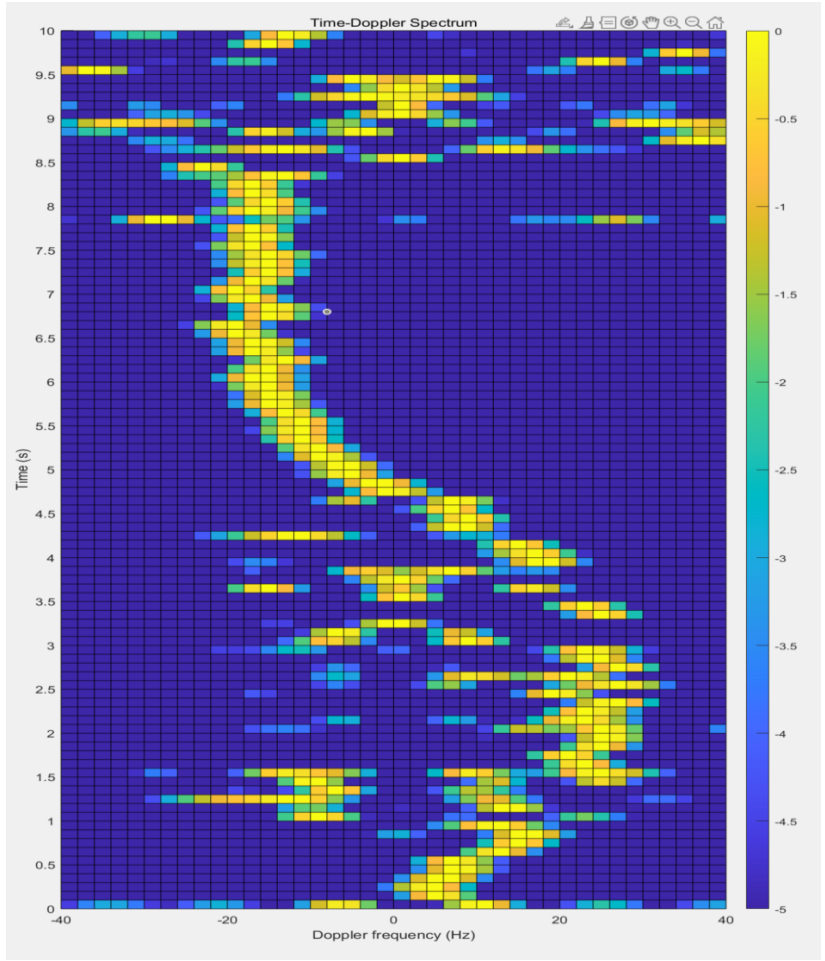
最后将代码中所有的duration除以5

作图



老师示例

老师的实例是以0.5s为滑动窗口每次滑动0.5s处理得到的10s数据的多普勒-时间谱图，时间段之间有重叠。
而我们是以0.1s为分析单位，每一个时间段之间没有重叠，因此所作图与示例有区别。
可以看到两图的走向和形状大致相同



实验作图

THANKS